

Prof. Dr. Antonio Marcos Selmini

Plataformas como Instagram, YouTube e LinkedIn exibem sugestões de nomes de usuários ou títulos enquanto você digita. Por baixo desse comportamento existe uma estrutura de busca eficiente que encontra, em tempo real, todos os termos que começam com o prefixo digitado.

Implementar um mini sistema de autocomplete usando uma BST (Binary Search Tree). O sistema deve armazenar nomes de usuários e, dado um prefixo, retornar todos os nomes que começam com esse prefixo em ordem alfabética.

- inserir(nome)**: insere um novo nome de usuário na BST.
- buscar(prefixo)**: percorre a BST e retorna uma lista com todos os nomes cujo início seja igual ao prefixo informado, em ordem alfabética.

Considere a seguinte lista de usuários cadastrados:

```
usuarios = ["mariana", "marcos", "mario", "ana", "marina", "bruna", "marcelo"]
```

Chamada	Saída esperada
buscar("mar")	['marcelo', 'marcos', 'mariana', 'marina', 'mario']
buscar("mari")	['mariana', 'marina', 'mario']
buscar("b")	['bruna']
buscar("z")	[]

- O percurso **in-order** garante que os nomes sejam visitados em ordem alfabética.
- Para verificar se um nome começa com o prefixo, use: **nome.startswith(prefixo)**.
- Normalize os dados ao inserir (ex: **.lower()**) para evitar problemas com maiúsculas / minúsculas.

Exchanges como Binance e Coinbase armazenam milhares de cotações por minuto. Para exibir gráficos e detectar máximas e mínimas em um intervalo de datas, precisam responder rapidamente a consultas do tipo: "qual foi o preço entre o dia X e o dia Y?"

Implementar uma BST onde cada nó armazena uma cotação com chave sendo o timestamp. O sistema deve suportar inserção de novas cotações e consulta eficiente por intervalo de datas (range query).

```
class No:
    def __init__(self, timestamp, preco):
        self.timestamp = timestamp
        self.preco      = preco
        self.esq        = None
        self.dir        = None
```

- a) **inserir(timestamp, preco)**: insere uma nova cotação na BST.
- b) **buscar(timestamp)**: retorna o preço exato para o timestamp informado, ou None.
- c) **range(inicio, fim)**: retorna lista de tuplas (timestamp, preco) em ordem crescente.

```
cotacoes = [
    (1000, 62400.0), # dia 1, 00:00
    (1100, 63100.5), # dia 1, 00:10
    (1200, 61800.0), # dia 1, 00:20
    (1300, 64500.0), # dia 1, 00:30
    (1400, 65200.0), # dia 1, 00:40
    (1500, 63900.0), # dia 1, 00:50
]
```

2

